

LEARNING RESOURCE GUIDE

Generative AI for Beginners | Professional Participant Workbook

Section	Section 3: Large Language Models (LLMs)
Lecture	Lecture 3.2: Transformers Architecture Explained

Learning Objectives

By the end of this lecture and its accompanying guide, you will be able to:

- Explain the core innovation of the Transformer architecture: self-attention and parallel processing.
- Describe the limitations of Recurrent Neural Networks (RNNs) that Transformers overcame.
- Articulate how self-attention allows a token to attend to all other tokens in a sequence simultaneously.
- Explain the roles of the encoder and decoder in the original Transformer architecture.
- Describe how multi-head attention enables the model to capture different types of relationships in text.

Introduction

The 2017 paper Attention Is All You Need introduced the Transformer architecture and, in doing so, made the modern AI era possible. Within five years of its publication, virtually every major language model — GPT, BERT, Claude, Gemini, LLaMA — was built on this architecture. Understanding why the Transformer was such a breakthrough, what problems it solved, and how its key mechanisms work is essential for understanding all of modern generative AI.

The key innovation of the Transformer is the attention mechanism — specifically, self-attention, which allows every token in a sequence to directly attend to every other token, regardless of their distance in the sequence. This solved the most critical limitation of the dominant pre-Transformer approach (Recurrent Neural Networks): the inability to model long-range dependencies efficiently. By processing all positions simultaneously rather than sequentially, Transformers also enabled parallelisation that dramatically accelerated training.

This lecture covers the fundamental mechanisms: the encoder-decoder structure of the original Transformer, the self-attention mechanism and how it works, multi-head attention that captures multiple types of relationships simultaneously, and why positional encoding is necessary when sequence information cannot be inferred from processing order. These concepts form the architectural foundation for the GPT models, BERT, and every subsequent major language model.

Core Concepts Explained

Limitations of RNNs and the Motivation for Transformers

Definition: Recurrent Neural Networks (RNNs) and their variants (LSTMs, GRUs) processed sequences token-by-token, maintaining a hidden state that carried information forward. This sequential processing caused two major problems: slow training (no parallelisation) and vanishing long-range dependencies (early context forgotten by the time later tokens are processed).

Business relevance: The sequential processing constraint of RNNs made training on large text corpora extremely slow, limiting the scale of models that could be trained in reasonable time. Transformers removed this constraint, enabling the training of models at the scale required for the emergent capabilities seen in GPT-3 and its successors.

Practitioner relevance: Understanding RNN limitations explains why all major modern language models are Transformer-based rather than RNN-based. When evaluating legacy AI systems, checking whether they use RNN-based architectures identifies a potential performance ceiling compared to Transformer-based alternatives.

Real-world example: An RNN processing the sentence The cat that had been sleeping peacefully on the warm mat suddenly jumped would need to carry information about cat through every subsequent token (that, had, been, sleeping, peacefully, on, the, warm, mat) before it reaches jumped. By the time it reaches jumped, the connection to cat is significantly diluted — making it difficult to correctly resolve that the cat is the subject of jumped.

Risks or limitations: Some organisations have deployed RNN-based production systems that are difficult to replace. Understanding RNN limitations helps quantify the performance gap and make the case for architecture migration where justified.

Self-Attention

Definition: Self-attention is the mechanism by which each token in a sequence computes its representation by attending to (weighting) all other tokens in the sequence simultaneously. Each token produces three vectors — Query, Key, and Value — and attention scores are computed by comparing each token's Query against all other tokens' Keys, then using the scores to weight a sum of Value vectors.

Business relevance: Self-attention is why modern language models can understand pronoun reference, subject-verb agreement across long distances, contextual word meaning (bank means something different in The river bank versus The bank balance), and semantic relationships across entire documents.

Practitioner relevance: Understanding attention allows practitioners to interpret what a model is doing when it generates text: each token's representation is a weighted combination of all other tokens, where the weights reflect learned relevance. Tools that visualise attention weights can help debug or understand model behaviour.

Real-world example: In the sentence The cat that had been sleeping peacefully on the warm mat suddenly jumped, when computing the representation of jumped, the self-attention mechanism assigns high attention weight to cat (the subject) even though they are eight tokens apart. This direct connection is what allows Transformers to handle long-range dependencies that RNNs struggled with.

Multi-Head Attention

Definition: Multi-head attention runs multiple self-attention computations (heads) in parallel, each with different weight matrices. This allows each head to learn to attend to different types of relationships — syntactic structure, semantic similarity, long-range coreference — simultaneously.

Business relevance: Multi-head attention is what gives Transformer models their rich contextual understanding. Different heads can specialise in different linguistic phenomena, collectively enabling the model to capture the full complexity of language in a single layer. This is why Transformers excel at tasks requiring nuanced language understanding.

Practitioner relevance: When analysing model behaviour, attention head specialisation can be analysed to understand what different heads have learned. Some research has found heads specialising in syntax (subject-

verb relationships), coreference (pronouns to referents), and positional patterns. This interpretability is more tractable than analysing individual weights.

Real-world example: In a model processing a legal contract, one attention head might specialise in linking pronouns to their antecedents (it refers to the Agreement from clause 1.3), another might track syntactic structure (verb-subject agreement), and another might attend to semantically similar terms across the document. Together, these heads produce a rich contextual representation.

Encoder, Decoder, and Encoder-Decoder Architectures

Definition: The original Transformer had both an encoder (which processes the input sequence into contextual representations) and a decoder (which generates the output sequence token by token, attending to encoder representations and previously generated tokens). Later architectures use encoder-only (BERT, for understanding tasks), decoder-only (GPT, for generation), or the original encoder-decoder (T5, for translation and summarisation).

Business relevance: The architecture choice — encoder-only, decoder-only, or encoder-decoder — determines what tasks the model is suited for. GPT-style decoder-only models are generalist generators; BERT-style encoder-only models are efficient classifiers and feature extractors. Knowing the architecture helps evaluate whether a model is appropriate for a given task.

Practitioner relevance: When evaluating language model products, checking whether they use an encoder-only, decoder-only, or encoder-decoder architecture immediately signals their intended use cases and capabilities. An encoder-only model cannot generate text; a decoder-only model cannot produce fixed-length encodings for similarity search without adaptation.

Real-world example: Google Translate uses an encoder-decoder architecture: the encoder processes the source language text into contextual representations, and the decoder generates the target language translation token by token while attending to the encoder's representation. GPT models use decoder-only architecture for pure generation without a separate encoding step.

Practical Applications

Document Understanding and Information Extraction

Encoder-only Transformer models like BERT and its variants are efficient at document classification, named entity recognition, question answering over documents, and semantic similarity. Because they process the full sequence bidirectionally (attending to both left and right context), they build richer representations than decoder-only models for pure understanding tasks.

Code Understanding and Generation

The same Transformer architecture that enables natural language processing — with attention across all tokens — also works on code. Models like GitHub Copilot and Claude use Transformer architectures to understand code context across a file or project and generate syntactically and semantically correct completions. The attention mechanism allows the model to connect a function call to its definition, even if they are thousands of tokens apart.

Machine Translation at Scale

Modern neural machine translation systems use Transformer encoder-decoder architectures and achieve quality that rivals human translators on many language pairs. The encoder builds a rich contextual representation of the

source sentence, and the decoder generates the target translation while attending to that representation — enabling both global coherence and local accuracy.

Best Practices

- Match Transformer architecture variant to task requirements. Use encoder-only models (BERT variants) for classification, extraction, and similarity tasks. Use decoder-only models (GPT variants) for text generation. Use encoder-decoder models (T5, BART) for translation and summarisation tasks where input and output sequences differ.
- Leverage attention visualisation for model debugging. When a Transformer model produces unexpected outputs, visualising attention weights can reveal whether the model is attending to the right tokens. This is more interpretable than examining raw weights and can guide prompt engineering or fine-tuning decisions.
- Consider context window limits in application design. Transformer attention scales quadratically with sequence length — processing a 4,000-token sequence requires four times the compute of a 2,000-token sequence. Design applications that fit within practical context window limits rather than assuming indefinite scaling.
- Evaluate model architecture before comparing benchmarks. Comparing a 7B-parameter encoder-only model to a 7B-parameter decoder-only model on a generation task is not a fair comparison — they are designed for different tasks. Architecture alignment with the task is as important as parameter count when interpreting benchmark results.

Common Mistakes and Misconceptions

Misconception: Assuming all Transformer models can do the same things because they share the Transformer architecture.

Correct approach: Encoder-only, decoder-only, and encoder-decoder Transformers have different architectural properties suited to different tasks. An encoder-only model cannot generate open-ended text; a decoder-only model is not optimal for producing fixed-length document embeddings for similarity search.

Misconception: Thinking attention mechanisms mean the model understands language in a human cognitive sense.

Correct approach: Attention is a weighted combination mechanism — a mathematical operation that allows tokens to influence each other's representations. It enables the model to capture contextual dependencies but does not constitute understanding in a cognitive sense.

Misconception: Ignoring context window limits when designing Transformer-based applications.

Correct approach: Transformers have a finite context window — the maximum number of tokens they can process in one pass. Input that exceeds this limit is either truncated or handled through chunking strategies. Application design must account for this constraint.

Reference Table: Transformer Architecture Variants — Use Case Reference

Use this table to identify which Transformer architecture variant is most appropriate for different NLP tasks.

Architecture	Examples	Best For	Not Ideal For
Encoder-only	BERT, RoBERTa, DeBERTa	Classification, extraction,	Open-ended text generation

		similarity, NLU	
Decoder-only	GPT-3/4, Claude, Llama, Mistral	Text generation, summarisation, Q&A, coding	Producing fixed-length embeddings without adaptation
Encoder-Decoder	T5, BART, mT5	Translation, summarisation, structured generation	Pure generation without distinct source/target
Sparse/Mixture-of-Experts	Mixtral, Switch Transformer	Scale efficiency, multi-task performance	Memory-constrained deployment

How to use this: When a vendor describes their model architecture, locate it in this table to immediately understand the intended use cases and limitations. Encoder-only vs decoder-only is as important a distinction as parameter count.

Knowledge Check

Test your understanding before moving on. Answers are shown below each question.

1. What was the primary training speed limitation of RNN-based language models that Transformers resolved?

- A. RNNs required labelled data; Transformers do not.
- B. RNNs processed sequences sequentially, preventing parallelisation; Transformers process all positions simultaneously.
- C. RNNs had more parameters than Transformers, making training slower.
- D. RNNs could not be trained on GPU hardware.

Answer: B. RNNs processed sequences sequentially, preventing parallelisation; Transformers process all positions simultaneously.

Why: Sequential processing in RNNs meant token N could not be processed until token $N-1$ was complete, preventing parallel computation. Transformers process all positions simultaneously using attention, enabling massive parallelisation on GPU hardware.

2. In a sentence like *The trophy didn't fit in the suitcase because it was too big*, the word it refers to trophy. How does the Transformer handle this long-range coreference?

- A. The model looks up the word it in a coreference database.
- B. Self-attention allows the representation of it to be computed with high attention weight assigned to trophy, directly linking them regardless of distance.
- C. The model processes the sentence sequentially and carries the reference forward through its hidden state.
- D. Multi-head attention averages all word representations equally.

Answer: B. Self-attention allows the representation of it to be computed with high attention weight assigned to trophy, directly linking them regardless of distance.

Why: Self-attention computes direct connections between any two tokens in the sequence, regardless of their distance. This allows the model to link pronouns to their antecedents even across long spans of text.

3. A team needs to build an AI system that classifies customer support tickets into twelve categories. Which Transformer architecture is most appropriate?

- A. Decoder-only (GPT-style) — because it has the most parameters.
- B. Encoder-only (BERT-style) — because it builds bidirectional contextual representations optimal for classification tasks.

C. Encoder-decoder (T5-style) — because it was designed for classification.

D. The architecture choice does not matter for classification tasks.

Answer: B. Encoder-only (BERT-style) — because it builds bidirectional contextual representations optimal for classification tasks.

Why: *Encoder-only models process the full sequence bidirectionally, building rich contextual representations ideal for classification tasks. Decoder-only models generate text sequentially and are not optimised for producing fixed-length classification outputs.*

Reflection Questions

1. Self-attention computes a weighted combination of all tokens based on relevance. Can you think of analogies in professional work where considering all available information simultaneously (rather than sequentially) would produce better judgements?
2. The Transformer was published as a single paper in 2017 and within five years underpinned virtually all frontier AI models. What does this suggest about how quickly the AI field can pivot when a significantly better approach is demonstrated?
3. Multi-head attention allows different heads to specialise in different linguistic relationships. How might this property be used to build more interpretable AI systems in regulated domains like healthcare or legal services?
4. The context window limit of Transformer models imposes a practical constraint on how much information they can process at once. How would you design an AI-assisted document review process for very long contracts or legal documents that exceed typical context window sizes?

Key Takeaways

- The Transformer architecture, introduced in 2017, is the foundation of virtually all modern large language models.
- Self-attention allows every token to attend to every other token simultaneously, enabling direct modelling of long-range dependencies without sequential processing.
- RNNs were limited by sequential processing (slow training) and vanishing long-range dependencies; Transformers solved both through parallel attention computation.
- Multi-head attention runs multiple attention computations in parallel, allowing each head to specialise in different types of linguistic relationships.
- Encoder-only models (BERT) are optimised for understanding tasks; decoder-only models (GPT) for generation; encoder-decoder models (T5) for translation and structured generation.
- Transformer attention scales quadratically with sequence length, imposing practical context window limits that application designers must account for.
- Positional encoding provides sequence order information to Transformers, which process all positions simultaneously and cannot infer order from processing sequence.

Recommended Next Actions

5. Sketch the high-level Transformer architecture: input embeddings, positional encoding, multi-head self-attention, feed-forward network, and output. Label each component.

6. Look up whether one AI tool you use regularly uses an encoder-only, decoder-only, or encoder-decoder architecture. Use the table from this guide to verify that the architecture is appropriate for how you use it.
7. Add to your glossary: Transformer, Self-Attention, Multi-Head Attention, Encoder, Decoder, Positional Encoding, Context Window, RNN, LSTM.
8. Write a one-paragraph explanation of why Transformers replaced RNNs for large-scale language modelling, suitable for a technical briefing to a non-specialist audience.